Question 1

```
from datasets import load_dataset
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Load dataset
dataset = load_dataset("scikit-learn/churn-prediction")
df = dataset["train"].to_pandas()

# Select predictors and target
X = df[["tenure", "MonthlyCharges"]]
y = df["Churn"].map({"No": 0, "Yes": 1})

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state = 42,
    stratify = y
)

# Fit logistic regression
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Confusion Matrix
print("Confusion Matix:")
print(confusion_matrix(y_test,y_pred))

# Perfomance metrics
print("\nPerfomance Metrics")
print("---------------------")

print("Accuracy:",accuracy_score(y_test,y_pred))
print("Precision:",precision_score(y_test,y_pred))
print("Recall:",recall_score(y_test,y_pred))
print("F1-score:",f1_score(y_test,y_pred))

# Complete classification report
print("\nClassification Report:")
print(classification_report(y_test,y_pred))
```

```
Confusion Matix:
[[931 104]
 [215 159]]

Perfomance Metrics
---------------------
Accuracy: 0.7735982966643009
Precision: 0.6045627376425855
Recall: 0.42513368983957217
F1-score: 0.49921507064364207

Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.90      0.85      1035
           1       0.60      0.43      0.50       374

    accuracy                           0.77      1409
   macro avg       0.71      0.66      0.68      1409
weighted avg       0.76      0.77      0.76      1409
```

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Predicted probabilities for churn
y_prob=model.predict_proba(X_test)[:,1]

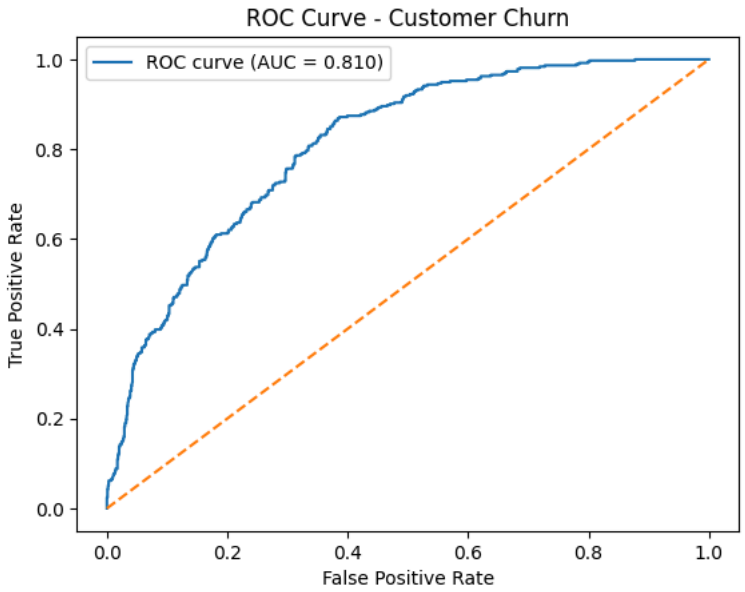
# ROC curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
```

```
# AUC
auc = roc_auc_score(y_test, y_prob)
print("AUC:",auc)

# Plot ROC curve
plt.plot(fpr, tpr, label=f"ROC curve (AUC = {auc:.3f})")
plt.plot([0, 1], [0,1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Customer Churn")
plt.legend()
plt.show()
```

```
AUC: 0.8100855098297552
```



Question 2

```
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Load dataset
dataset = sns.load_dataset("penguins")

df = dataset.copy()

df = dataset[dataset["species"].isin(["Adelie", "Gentoo"])].copy()

features = [
    "bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"
]

df = df[features + ["species"]].dropna()

X = df[features]
y = (df["species"] == "Gentoo").astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.2,
    random_state=42,
    stratify=y
)

# standardize
model = Pipeline([
    ("scaler", StandardScaler()),
    ("logistic", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]


# confusion matrix
print(f"Confusion Matrix: \n{confusion_matrix(y_test, y_pred)}")
```

```
# performance metrics
print(f"\nPerformance Metrics: \nAccuracy: {accuracy_score(y_test, y_pred)}")
print(f"Precision: {precision_score(y_test, y_pred)}")
print(f"Recall: {recall_score(y_test, y_pred)}")
print(f"F1 Score: {f1_score(y_test, y_pred)}")

# classification report
print(f"\nClassification Report: \n{classification_report(y_test, y_pred)}\n")
print(f"\nInterpretation: \nThus, the model is correct 100% of times.")
```

```
Confusion Matrix:
[[30  0]
 [ 0 25]]

Performance Metrics:
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        30
           1       1.00      1.00      1.00        25

    accuracy                           1.00        55
   macro avg       1.00      1.00      1.00        55
weighted avg       1.00      1.00      1.00        55



Interpretation:
Thus, the model is correct 100% of times.
```

```
# roc curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)

# auc
auc = roc_auc_score(y_test, y_prob)

print("AUC: ",auc)

# plot roc curve
plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {auc: .3f})")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Adelie/Gentoo")
plt.legend()
plt.show()
```

```
AUC:  1.0
```

